The **syntax of the SQL programming language** is defined and maintained by ISO/IEC SC 32 as part of *ISO/IEC 9075*. This standard is not freely available. Despite the existence of the standard, SQL code is not completely portable among different database systems without adjustments.

Language elements

}statement

- are words that are defined in the SQL language. They are either reserved (e.g. SELECT, COUNT and YEAR), or non-reserved (e.g. ASC, DOMAIN and KEY). List of SQL reserved words.
- *Identifiers* are names on database objects, like tables, columns and schemas. An identifier may not be equal to a reserved keyword, unless it is a delimited identifier. Delimited identifiers means identifiers enclosed in double quotation marks. They can contain characters normally not supported in SQL identifiers, and they can be identical to a reserved word, e.g. a column named YEAR is specified as "YEAR".
    - In MySQL, double quotes are string literal delimiters by default instead. Enabling the ansi_quotes SQL mode enforces the SQL standard behavior. These can also be used regardless of this mode through backticks: `YEAR`.
- *Clauses*, which are constituent components of statements and queries. (In some cases, these are optional.)[1]
- *Expressions*, which can produce either scalar values, or tables consisting of columns and rows of data
- *Predicates*, which specify conditions that can be evaluated to SQL three-valued logic (3VL) (true/false/unknown) or Boolean truth values and are used to limit the effects of statements and queries, or to change program flow.
- *Queries*, which retrieve the data based on specific criteria. This is an important element of *SQL*.
- *Statements*, which may have a persistent effect on schemata and data, or may control transactions, program flow, connections, sessions, or diagnostics.
    - SQL statements also include the semicolon (";") statement terminator. Though not required on every platform, it is defined as a standard part of the SQL grammar.
- *Insignificant whitespace* is generally ignored in SQL statements and queries, making it easier to format SQL code for readability.

Operators

Operator	Description	Example
=	Equal to	Author = 'Alcott'
<>	Not equal to (many dialects also accept !=)	Dept <> 'Sales'
>	Greater than	Hire_Date > '2012-01-31'
<	Less than	Bonus < 50000.00
>=	Greater than or equal	Dependents >= 2
<=	Less than or equal	Rate <= 0.05
[NOT] BETWEEN [SYMMETRIC]	Between an inclusive range. SYMMETRIC inverts the range bounds if the first is higher than the second.	Cost **BETWEEN** 100.00 **AND** 500.00
[NOT] LIKE [ESCAPE]	Begins with a character pattern	Full_Name **LIKE** 'Will%'
	Contains a character pattern	Full_Name **LIKE** '%Will%'

[NOT] IN	Equal to one of multiple possible values	DeptCode **IN** (101, 103, 209)
IS [NOT] NULL	Compare to null (missing data)	Address **IS NOT NULL**
IS [**NOT**] **TRUE** *or* **IS** [**NOT**] **FALSE**	Boolean truth value test	PaidVacation **IS TRUE**
IS NOT DISTINCT FROM	Is equal to value or both are nulls (missing data)	Debt **IS NOT DISTINCT FROM** - Receivables
AS	Used to change a column name when viewing results	**SELECT** employee **AS** department1

Other operators have at times been suggested or implemented, such as the skyline operator (for finding only those rows that are not 'worse' than any others).

SQL has the case expression, which was introduced in SQL-92. In its most general form, which is called a "searched case" in the SQL standard:

```
CASE WHEN n > 0
        THEN 'positive'
     WHEN n < 0
        THEN 'negative'
     ELSE 'zero'
END
```

SQL tests WHEN conditions in the order they appear in the source. If the source does not specify an ELSE expression, SQL defaults to ELSE NULL. An abbreviated syntax called "simple case" can also be used:

CASE n **WHEN** 1

THEN 'One'

WHEN 2

THEN 'Two'

ELSE 'I cannot count that high'

END

This syntax uses implicit equality comparisons, with the usual caveats for comparing with NULL.

There are two short forms for special CASE expressions: COALESCE and NULLIF.

The COALESCE expression returns the value of the first non-NULL operand, found by working from left to right, or NULL if all the operands equal NULL.

COALESCE(x1,x2)

is equivalent to:

CASE WHEN x1 **IS NOT NULL THEN** x1

ELSE x2

END

The NULLIF expression has two operands and returns NULL if the operands have the same value, otherwise it has the value of the first operand.

NULLIF(x1, x2)

is equivalent to

CASE WHEN x1 = x2 **THEN NULL ELSE** x1 **END**

Comments

Standard SQL allows two formats for comments: -- comment, which is ended by the first newline, and /* comment */, which can span multiple lines.

Queries

The most common operation in SQL, the query, makes use of the declarative SELECT statement. SELECT retrieves data from one or more tables, or expressions. Standard SELECT statements have no persistent effects on the database. Some non-standard

implementations of SELECT can have persistent effects, such as the SELECT INTO syntax provided in some databases.[2]

Queries allow the user to describe desired data, leaving the database management system (DBMS) to carry out planning, optimizing, and performing the physical operations necessary to produce that result as it chooses.

A query includes a list of columns to include in the final result, normally immediately following the SELECT keyword. An asterisk ("*"), or "wildcard", can be used to specify that the query should return all columns of the queried tables. SELECT is the most complex statement in SQL, with optional keywords and clauses that include:

- The FROM clause, which indicates the table(s) to retrieve data from.[3]:400 The FROM clause can include optional JOIN subclauses to specify the rules for joining tables.
- The WHERE clause includes a condition, which restricts the rows returned by the query. The WHERE clause eliminates all rows from the result set where the condition does not evaluate to True.[3]:402
- The GROUP BY clause projects rows having common values into a smaller set of rows.[*clarification needed*] GROUP BY is often used in conjunction with SQL aggregation functions or to eliminate duplicate rows from a result set. The WHERE clause is applied before the GROUP BY clause.
- The HAVING clause includes a predicate used to filter rows resulting from the GROUP BY clause. Because it acts on the results of the GROUP BY clause, aggregation functions can be used in the HAVING clause predicate.
- The ORDER BY clause identifies which column[s] to use to sort the resulting data, and in which direction to sort them (ascending or descending). Without an ORDER BY clause, the order of rows returned by an SQL query is undefined.
- The DISTINCT keyword eliminates duplicate data.[4]
- The OFFSET clause specifies the number of rows to skip before starting to return data.
- The FETCH FIRST clause specifies the number of rows to return. Some SQL databases instead have non-standard alternatives, e.g. LIMIT, TOP or ROWNUM.

The clauses of a query have a particular order of execution,[5] which is denoted by the number on the right hand side. It is as follows:

SELECT *<columns>*	5.
FROM *<table>*	1.
WHERE *<predicate on rows>*	2.
GROUP BY *<columns>*	3.
HAVING *<predicate on groups>*	4.
ORDER BY *<columns>*	6.
OFFSET	7.
FETCH FIRST	8.

The following example of a SELECT query returns a list of expensive books. The query retrieves all rows from the *Book* table in which the *price* column contains a value greater than 100.00. The result is sorted in ascending order by *title*. The asterisk (*) in the *select list* indicates that all columns of the *Book* table should be included in the result set.

```
SELECT *
 FROM  Book
 WHERE price > 100.00
 ORDER BY title;
```

The example below demonstrates a query of multiple tables, grouping, and aggregation, by returning a list of books and the number of authors associated with each book.

```
SELECT Book.title AS Title,
    count(*) AS Authors
 FROM  Book
 JOIN  Book_author
   ON  Book.isbn = Book_author.isbn
 GROUP BY Book.title;
```

Example output might resemble the following:

Title	Authors

---------- -------

SQL Examples and Guide 4

The Joy of SQL 1

An Introduction to SQL 2

Pitfalls of SQL 1

Under the precondition that *isbn* is the only common column name of the two tables and that a column named *title* only exists in the *Book* table, one could re-write the query above in the following form:

```
SELECT title,
    count(*) AS Authors
 FROM  Book
 NATURAL JOIN Book_author
 GROUP BY title;
```

However, many[*quantify*] vendors either do not support this approach, or require certain column-naming conventions for natural joins to work effectively.

SQL includes operators and functions for calculating values on stored values. SQL allows the use of expressions in the *select list* to project data, as in the following example, which returns a list of books that cost more than 100.00 with an additional *sales_tax* column containing a sales tax figure calculated at 6% of the *price*.

```
SELECT isbn,
    title,
    price,
    price * 0.06 AS sales_tax
 FROM  Book
 WHERE price > 100.00
 ORDER BY title;
```

Subqueries

Queries can be nested so that the results of one query can be used in another query via a relational operator or aggregation function. A nested query is also known as a *subquery*. While joins and other table operations provide computationally superior (i.e. faster) alternatives in many cases, the use of subqueries introduces a hierarchy in execution that can be useful or necessary. In the following example, the aggregation function AVG receives as input the result of a subquery:

```
SELECT isbn,
    title,
    price
FROM  Book
WHERE price < (SELECT AVG(price) FROM Book)
ORDER BY title;
```

A subquery can use values from the outer query, in which case it is known as a correlated subquery.

Since 1999 the SQL standard allows WITH clauses for subqueries, i.e. named subqueries, usually called common table expressions (also called subquery factoring). CTEs can also be recursive by referring to themselves; the resulting mechanism allows tree or graph traversals (when represented as relations), and more generally fixpoint computations.

Derived table

A *derived table* is the use of referencing an SQL subquery in a FROM clause. Essentially, the derived table is a subquery that can be selected from or joined to. The derived table functionality allows the user to reference the subquery as a table. The derived table is sometimes referred to as an *inline view* or a *subselect*.

In the following example, the SQL statement involves a join from the initial "Book" table to the derived table "sales". This derived table captures associated book sales information using the ISBN to join to the "Book" table. As a result, the derived table provides the result set with additional columns (the number of items sold and the company that sold the books):

```
SELECT b.isbn, b.title, b.price, sales.items_sold, sales.company_nm
FROM Book b
  JOIN (SELECT SUM(Items_Sold) Items_Sold, Company_Nm, ISBN
      FROM Book_Sales
```

GROUP BY Company_Nm, ISBN) sales

ON sales.isbn = b.isbn

Null or three-valued logic (3VL)

Main article: Null (SQL)

The concept of Null allows SQL to deal with missing information in the relational model. The word NULL is a reserved keyword in SQL, used to identify the Null special marker. Comparisons with Null, for instance equality (=) in WHERE clauses, results in an Unknown truth value. In SELECT statements SQL returns only results for which the WHERE clause returns a value of True; i.e., it excludes results with values of False and also excludes those whose value is Unknown.

Along with True and False, the Unknown resulting from direct comparisons with Null thus brings a fragment of three-valued logic to SQL. The truth tables SQL uses for AND, OR, and NOT correspond to a common fragment of the Kleene and Lukasiewicz three-valued logic (which differ in their definition of implication, however SQL defines no such operation).[6]

p AND q		p		
		True	**False**	**Unknown**
q	**True**	True	False	Unknown
	False	False	False	False
	Unknown	Unknown	False	Unknown

p OR q		p		
		True	**False**	**Unknown**
q	**True**	True	True	True
	False	True	False	Unknown
	Unknown	True	Unknown	Unknown

p = q		p		
		True	**False**	**Unknown**
q	**True**	True	False	Unknown
	False	False	True	Unknown
	Unknown	Unknown	Unknown	Unknown

q	NOT q
True	False
False	True
Unknown	Unknown

There are however disputes about the semantic interpretation of Nulls in SQL because of its treatment outside direct comparisons. As seen in the table above, direct equality comparisons between two NULLs in SQL (e.g. NULL = NULL) return a truth value of Unknown. This is in line with the interpretation that Null does not have a value (and is not a member of any data

domain) but is rather a placeholder or "mark" for missing information. However, the principle that two Nulls aren't equal to each other is effectively violated in the SQL specification for the UNION and INTERSECT operators, which do identify nulls with each other.[7] Consequently, these set operations in SQL may produce results not representing sure information, unlike operations involving explicit comparisons with NULL (e.g. those in a WHERE clause discussed above). In Codd's 1979 proposal (which was basically adopted by SQL92) this semantic inconsistency is rationalized by arguing that removal of duplicates in set operations happens "at a lower level of detail than equality testing in the evaluation of retrieval operations".[6] However, computer-science professor Ron van der Meyden concluded that "The inconsistencies in the SQL standard mean that it is not possible to ascribe any intuitive logical semantics to the treatment of nulls in SQL."[7]

Additionally, because SQL operators return Unknown when comparing anything with Null directly, SQL provides two Null-specific comparison predicates: IS NULL and IS NOT NULL test whether data is or is not Null.[8] SQL does not explicitly support universal quantification, and must work it out as a negated existential quantification.[9][10][11] There is also the <row value expression> IS DISTINCT FROM <row value expression> infixed comparison operator, which returns TRUE unless both operands are equal or both are NULL. Likewise, IS NOT DISTINCT FROM is defined as NOT (<row value expression> IS DISTINCT FROM <row value expression>). SQL:1999 also introduced BOOLEAN data type, which according to the standard can also hold Unknown values if it is nullable. In practice, a number of systems (e.g. PostgreSQL) implement the BOOLEAN Unknown as a BOOLEAN NULL, which the standard says that the NULL BOOLEAN and UNKNOWN "may be used interchangeably to mean exactly the same thing".[12][13]

Data manipulation

The Data Manipulation Language (DML) is the subset of SQL used to add, update and delete data:

- INSERT adds rows (formally tuples) to an existing table, e.g.:

INSERT INTO example

(column1, column2, column3)

VALUES

('test', 'N', **NULL**);

- UPDATE modifies a set of existing table rows, e.g.:

UPDATE example

SET column1 = 'updated value'

WHERE column2 = 'N';

- DELETE removes existing rows from a table, e.g.:

DELETE FROM example

WHERE column2 = 'N';

- MERGE is used to combine the data of multiple tables. It combines the INSERT and UPDATE elements. It is defined in the SQL:2003 standard; prior to that, some databases provided similar functionality via different syntax, sometimes called "upsert".

MERGE INTO table_name **USING** table_reference **ON** (condition)

WHEN MATCHED THEN

UPDATE SET column1 = value1 [, column2 = value2 ...]

WHEN NOT MATCHED THEN

INSERT (column1 [, column2 ...]) **VALUES** (value1 [, value2 ...])

Transaction controls

Transactions, if available, wrap DML operations:

- START TRANSACTION (or BEGIN WORK, or BEGIN TRANSACTION, depending on SQL dialect) marks the start of a database transaction, which either completes entirely or not at all.
- SAVE TRANSACTION (or SAVEPOINT) saves the state of the database at the current point in transaction

CREATE TABLE tbl_1(id int);

INSERT INTO tbl_1(id) **VALUES**(1);

INSERT INTO tbl_1(id) **VALUES**(2);

COMMIT;

UPDATE tbl_1 **SET** id=200 **WHERE** id=1;

SAVEPOINT id_1upd;

UPDATE tbl_1 **SET** id=1000 **WHERE** id=2;

ROLLBACK to id_1upd;

SELECT id **from** tbl_1;

- COMMIT makes all data changes in a transaction permanent.
- ROLLBACK discards all data changes since the last COMMIT or ROLLBACK, leaving the data as it was prior to those changes. Once the COMMIT statement completes, the transaction's changes cannot be rolled back.

COMMIT and ROLLBACK terminate the current transaction and release data locks. In the absence of a START TRANSACTION or similar statement, the semantics of SQL are implementation-dependent. The following example shows a classic transfer of funds transaction, where money is removed from one account and added to another. If either the removal or the addition fails, the entire transaction is rolled back.

START TRANSACTION;

UPDATE Account **SET** amount=amount-200 **WHERE** account_number=1234;

UPDATE Account **SET** amount=amount+200 **WHERE** account_number=2345;

IF ERRORS=0 **COMMIT**;

IF ERRORS<>0 **ROLLBACK**;

Data definition

The Data Definition Language (DDL) manages table and index structure. The most basic items of DDL are the CREATE, ALTER, RENAME, DROP and TRUNCATE statements:

- CREATE creates an object (a table, for example) in the database, e.g.:

CREATE TABLE example(

column1 INTEGER,

column2 VARCHAR(50),

column3 DATE **NOT NULL**,

PRIMARY KEY (column1, column2)

);

- ALTER modifies the structure of an existing object in various ways, for example, adding a column to an existing table or a constraint, e.g.:

ALTER TABLE example **ADD** column4 INTEGER **DEFAULT** 25 **NOT NULL**;

- TRUNCATE deletes all data from a table in a very fast way, deleting the data inside the table and not the table itself. It usually implies a subsequent COMMIT operation, i.e., it cannot be rolled back (data is not written to the logs for rollback later, unlike DELETE).

TRUNCATE TABLE example;

- DROP deletes an object in the database, usually irretrievably, i.e., it cannot be rolled back, e.g.:

DROP TABLE example;

Data types

Each column in an SQL table declares the type(s) that column may contain. ANSI SQL includes the following data types.[14]

Character strings and national character strings

- CHARACTER(*n*) (or CHAR(*n*)): fixed-width *n*-character string, padded with spaces as needed
- CHARACTER VARYING(*n*) (or VARCHAR(*n*)): variable-width string with a maximum size of *n* characters
- CHARACTER LARGE OBJECT(*n*[K|M|G|T]) (or CLOB(*n*[K|M|G|T])): character large object with a maximum size of *n*[K|M|G|T] characters
- NATIONAL CHARACTER(*n*) (or NCHAR(*n*)): fixed width string supporting an international character set
- NATIONAL CHARACTER VARYING(*n*) (or NVARCHAR(*n*)): variable-width NCHAR string
- NATIONAL CHARACTER LARGE OBJECT(*n*[K|M|G|T]) (or NCLOB(*n*[K|M|G|T])): national character large object with a maximum size of *n*[K|M|G|T] characters

For the CHARACTER LARGE OBJECT and NATIONAL CHARACTER LARGE OBJECT data types, the multipliers K (1024), M (1048576), G (1073741824) and T (1099511627776) can be optionally used when specifying the length.

Binary

- BINARY(*n*): Fixed length binary string, maximum length *n*.
- BINARY VARYING(*n*) (or VARBINARY(*n*)): Variable length binary string, maximum length *n*.

- BINARY LARGE OBJECT(*n*[K|M|G|T]) (or BLOB(*n*[K|M|G|T])): binary large object with a maximum length *n*[K|M|G|T].

For the BINARY LARGE OBJECT data type, the multipliers K (1024), M (1048576), G (1073741824) and T (1099511627776) can be optionally used when specifying the length.

Boolean

- BOOLEAN

The BOOLEAN data type can store the values TRUE and FALSE.

Numerical

- INTEGER (or INT), SMALLINT and BIGINT
- FLOAT, REAL and DOUBLE PRECISION
- NUMERIC(*precision*, *scale*) or DECIMAL(*precision*, *scale*)
- DECFLOAT(*precision*)

For example, the number 123.45 has a precision of 5 and a scale of 2. The *precision* is a positive integer that determines the number of significant digits in a particular radix (binary or decimal). The *scale* is a non-negative integer. A scale of 0 indicates that the number is an integer. For a decimal number with scale S, the exact numeric value is the integer value of the significant digits divided by 10^S.

SQL provides the functions CEILING and FLOOR to round numerical values. (Popular vendor specific functions are TRUNC (Informix, DB2, PostgreSQL, Oracle and MySQL) and ROUND (Informix, SQLite, Sybase, Oracle, PostgreSQL, Microsoft SQL Server and Mimer SQL.))

Temporal (datetime)

- DATE: for date values (e.g. 2011-05-03).
- TIME: for time values (e.g. 15:51:36).
- TIME WITH TIME ZONE: the same as TIME, but including details about the time zone in question.
- TIMESTAMP: This is a DATE and a TIME put together in one value (e.g. 2011-05-03 15:51:36.123456).

- TIMESTAMP WITH TIME ZONE: the same as TIMESTAMP, but including details about the time zone in question.

The SQL function EXTRACT can be used for extracting a single field (seconds, for instance) of a datetime or interval value. The current system date / time of the database server can be called by using functions like CURRENT_DATE, CURRENT_TIMESTAMP, LOCALTIME, or LOCALTIMESTAMP. (Popular vendor specific functions are TO_DATE, TO_TIME, TO_TIMESTAMP, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, DAYOFYEAR, DAYOFMONTH and DAYOFWEEK.)

Interval (datetime)

- YEAR(*precision*): a number of years
- YEAR(*precision*) TO MONTH: a number of years and months
- MONTH(*precision*): a number of months
- DAY(*precision*): a number of days
- DAY(*precision*) TO HOUR: a number of days and hours
- DAY(*precision*) TO MINUTE: a number of days, hours and minutes
- DAY(*precision*) TO SECOND(*scale*): a number of days, hours, minutes and seconds
- HOUR(*precision*): a number of hours
- HOUR(*precision*) TO MINUTE: a number of hours and minutes
- HOUR(*precision*) TO SECOND(*scale*): a number of hours, minutes and seconds
- MINUTE(*precision*): a number of minutes
- MINUTE(*precision*) TO SECOND(*scale*): a number of minutes and seconds

Data control

The Data Control Language (DCL) authorizes users to access and manipulate data. Its two main statements are:

- GRANT authorizes one or more users to perform an operation or a set of operations on an object.
- REVOKE eliminates a grant, which may be the default grant.

Example:

GRANT SELECT, **UPDATE**

ON example

TO some_user, another_user;

REVOKE SELECT, **UPDATE**

ON example

FROM some_user, another_user;